

1. Algoritmo y Analisis de complejidad

1.1. Algoritmo de Backtracking

El algoritmo propuesto recibe la lista de jugadores (A /Universo) y los posibles subconjuntos a armar en base a esos jugadores (B /subconjuntos). En base a esto calculamos recursivamente el menor equipo posible que contenga al menos uno de los jugadores de cada equipo (B).

Implementamos las siguientes podas a las ramas de todas las soluciones posibles:

Cota superior: en base a un Hitting Set valido de tamaño reducido (solucion optima) podemos utilizarlo como una cota superior restrictiva que nos permite descartar ramas de caminos suboptimos sin tener la necesidad de recorrerlos completamente hasta las hojas.

Cobertura Previa: si el subconjunto parcial ya satisface la restricción actual, es decir si tenemos un elemento en el subconjunto parcial que es parte de la restricción actual seguimos hacia la proxima restricción sin la necesidad de explorar todas las combinaciones/ramificaciones posibles en base a todos los elementos de la restricción. Es decir, el algoritmo avanza en la profundidad del árbol de manera directa sin necesidad de abrir nuevos bucles de ramificación.

1.2. Pseudocodigo del algoritmo

```

1
2 def _BThittingSetOptimo(A,sets,indice,parcial,set_p,optimo):
3
4     if len(parcial)>=len(optimo):
5         return optimo
6     if len(sets)==indice:
7         return parcial[:]
8     b=sets[indice]
9     if any(x in set_p for x in b):
10         return _BThittingSetOptimo(A,sets,indice+1,parcial,set_p,optimo)
11     for x in b:
12         if x not in set_p:
13             parcial.append(x)
14             set_p.add(x)
15             optimo=_BThittingSetOptimo(A,sets,indice+1,parcial,set_p,optimo)
16             parcial.pop()
17             set_p.remove(x)
18     return optimo
19
20 def hittingSetOptimo(jugadores, equipos):
21
22     indice_inicial = 0
23     parcial_inicial = []
24     optimo_inicial = list(jugadores)
25
26     return _BThittingSetOptimo(jugadores, equipos, indice_inicial, parcial_inicial,
27                               set(), optimo_inicial)

```

1.3. Analisis de Complejidad

1.3.1. Complejidad del algoritmo de Backtracking

Tenemos dos variables A (jugadores) de n elementos y B (equipos) de m subconjuntos:

En el peor caso recorreremos los m subconjuntos y todas sus ramificaciones posibles. A continuación se detalla el análisis asintótico de este comportamiento:

Complejidad Temporal:

N =: El universo total de jugadores disponibles.

M =: El número total de conjuntos (equipos) que se deben *hitear*.

K =: El tamaño máximo de un equipo individual (es decir, $\max(\text{len}(b))$ para todo $b \in \text{equipos}$), donde $K \leq N$.

El algoritmo avanza de forma lineal sobre los equipos usando el parámetro **índice** (que va de 0 a M). Por cada equipo, si ese equipo ya está cubierto por el *hitting set* entonces continuo mi recursión hacia el próximo equipo. En caso contrario, recorro el equipo y por cada jugador no perteneciente al *hitting set* realizo una llamada recursiva incluyéndolo.

En el peor de los casos (ningún jugador del equipo pertenece al *Hitting Set*), se realizan tantas llamadas recursivas como elementos tenga el equipo. Es decir, hasta K ramificaciones.

En el peor escenario teórico (donde los subconjuntos son disjuntos y las podas no logran actuar tempranamente), el árbol tiene una profundidad de M niveles, y cada nivel se puede ramificar en K nuevos caminos. El número máximo de nodos visitados en el árbol será de orden:

$$\mathcal{O}(K^M)$$

Análisis del costo de las Operaciones por subconjunto:

En cada llamada de la función `_BhittingSetOptimo`, ocurren las siguientes operaciones:

- **Casos base y podas:** `len(parcial) >= len(optimo)` y `len(sets) == indice`. Son comparaciones $\mathcal{O}(1)$.
- **Verificación de cobertura:** `any(x in set_p for x in b)`. Como es un *set* cuesta $\mathcal{O}(1)$. Iterar sobre los elementos del equipo actual b toma $\mathcal{O}(K)$ en el peor de los casos.
- **Bucle For:** `for x in b:`. Itera un máximo de K veces realizando operaciones $\mathcal{O}(1)$.

Por lo tanto, el costo de procesamiento propio en cada subconjunto del árbol de recursión es $\mathcal{O}(K)$.

Complejidad Temporal Total:

Multiplicando el número total de subconjuntos por el costo de procesamiento dentro de cada uno, obtenemos la complejidad total en el peor de los casos:

$$\text{Complejidad Temporal} = \mathcal{O}(K \cdot K^M) = \mathcal{O}(K^{M+1})$$

1.4. Análisis tiempos

1.4.1. Variación de N con M fijo

Sets Disjuntos: Este escenario representa el caso frontera del problema, diseñado específicamente para anular la efectividad de las podas implementadas. Al no existir intersección de elementos entre las restricciones, el algoritmo se ve forzado a explorar la totalidad del árbol de decisiones para evaluar las combinaciones posibles de subconjuntos.

Para construir estas instancias, se distribuyen los N jugadores en M sets estrictamente disjuntos. Al ser la intersección entre cualquier par de conjuntos igual al conjunto vacío ($B_i \cap B_j = \emptyset, \forall i \neq j$), la condición de cobertura previa nunca se cumple en las primeras etapas de la recursión y la cota superior no logra actuar tempranamente.

Este comportamiento acentúa la naturaleza intrínseca del problema NP-completo, reflejando empíricamente un orden de complejidad exponencial respecto al tamaño del universo de jugadores (N).

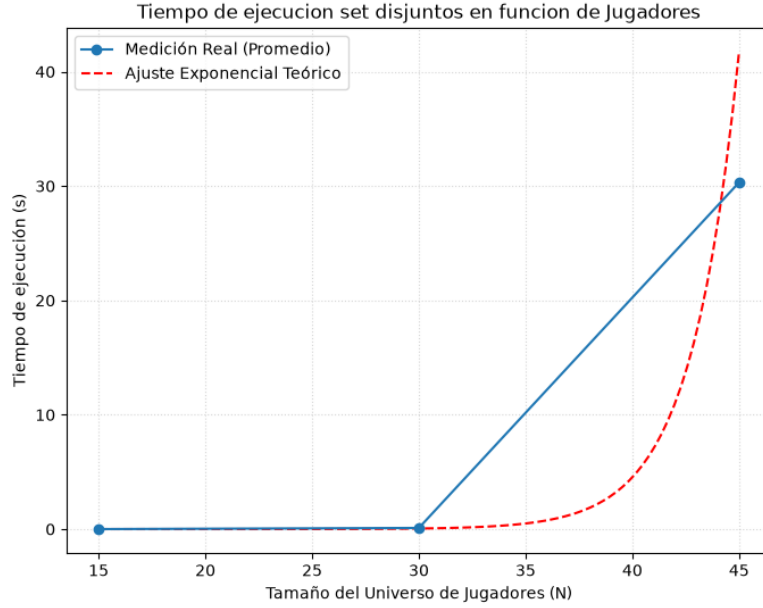


Figura 1: Tiempo de ejecución en función de la cantidad de jugadores N con M fijo.

El modelo de ajuste teórico correspondiente, calculado mediante el método de cuadrados mínimos, responde a la siguiente función:

$$T(n) = c_1 \cdot (c_2)^n$$

Donde c_1 representa la constante de proporcionalidad ligada al overhead de la máquina y las operaciones base, mientras que c_2 determina la tasa de ramificación real que experimenta el árbol de exploración de Backtracking.

Sets No Disjuntos: A diferencia del escenario disjunto, la utilización de instancias con sets no disjuntos representa un caso promedio o favorable, donde los conjuntos de restricciones comparten una cantidad significativa de elementos comunes ($B_i \cap B_j \neq \emptyset$ para diversos pares i, j). Este solapamiento estructural provee un entorno óptimo para la activación de las reglas de poda implementadas.

En primer lugar, la alta densidad de elementos repetidos incrementa drásticamente la efectividad de la poda por cobertura previa. Durante la exploración del árbol, la condición que evalúa si el subconjunto parcial ya satisface la restricción actual tiende a dar verdadero con frecuencia, permitiendo al algoritmo avanzar en la profundidad del árbol de manera directa sin necesidad de abrir nuevos bucles de ramificación.

En segundo lugar, la interconectividad de los conjuntos facilita el hallazgo muy temprano de un Hitting Set válido de tamaño reducido (óptimo global). Una vez establecida esta cota superior restrictiva, la poda por cota superior intercepta masivamente los caminos subóptimos, descartando ramas enteras mucho antes de llegar a las hojas del árbol.

Como consecuencia de este doble mecanismo de filtrado, el espacio de búsqueda efectivamente explorado se contrae de forma drástica. La curva de tiempo experimenta un comportamiento fuertemente linealizado respecto a N , desvinculándose de la tendencia exponencial teórica del peor caso.

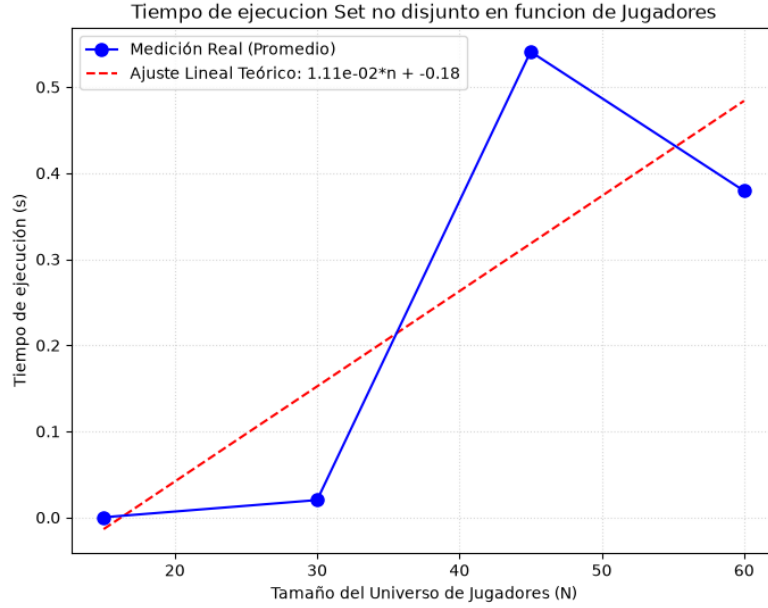


Figura 2: Tiempo de ejecución en función de la cantidad de jugadores N con M fijo.

El modelo de ajuste teórico correspondiente, calculado mediante el método de cuadrados mínimos, responde a la siguiente función:

$$T(n) = c_1 \cdot n + c_2$$

Donde c_1 representa la constante de proporcionalidad ligada al overhead de la máquina y las operaciones base, mientras que c_2 determina la tasa de ramificación real que experimenta el árbol de exploración de Backtracking.

1.4.2. Variacion de M con N fija

En este escenario se observa un fenómeno del tiempo de ejecución de crecimiento al principio, pero a medida que aumenta el número de restricciones, M , el crecimiento se frena y se estabiliza. En las instancias donde $M < N$, el incremento de restricciones añade complejidad al árbol al forzar la búsqueda de nuevos elementos de cobertura. Sin embargo, una vez alcanzado el umbral donde $M \geq N$, la alta redundancia de los conjuntos no disjuntos estabiliza el tamaño del Hitting Set óptimo global. Esto desencadena una activación prematura y masiva de la poda por cota superior, cortando la expansión del árbol combinatorio y provocando que el tiempo de cómputo se amesete de forma asintótica.

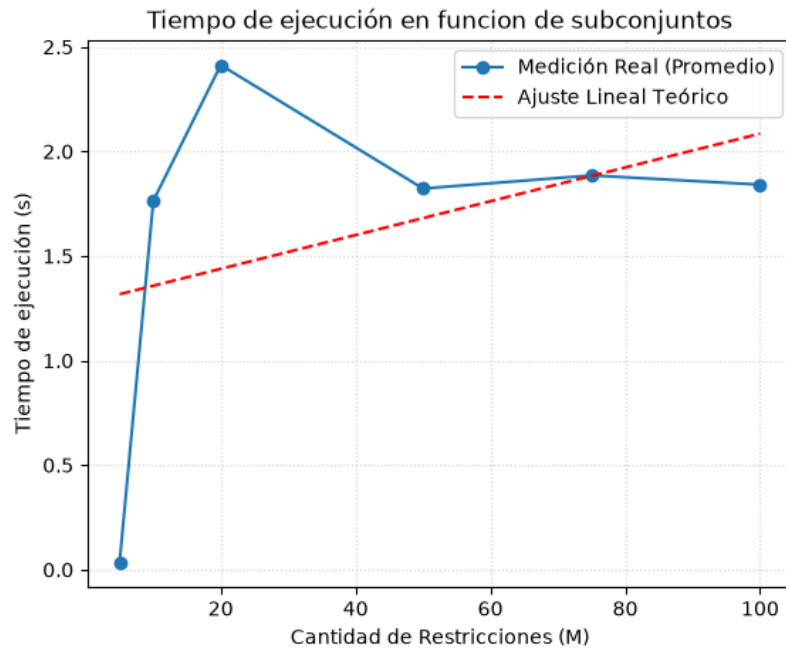


Figura 3: Tiempo de ejecución en función de la cantidad de restricciones M para un universo N fijo.

El modelo de ajuste teórico correspondiente, calculado mediante el método de cuadrados mínimos, responde a la siguiente función:

$$T(m) = c_1 \cdot m + c_2$$